AI-in-the-loop in Software Project Cycle Course Project CP 1

Group 22: 劉維凡、徐畹茜、李尹瑄、莊蕓安

1. Team Charter

Our agreed working norms.

- How we will make decisions when there is disagreement
    - For general project decisions, team members will discuss the available options and try to reach a consensus. If no agreement can be reached, the team will make a decision by majority vote.
    - If team members disagree about whether AI-generated code is good enough to merge, we will evaluate it based on whether it passes the existing tests, passes the PR review checklist, can be understood and explained by another team member, and does not introduce known security or functionality issues.
    - If the disagreement is about coding style or personal preference, we will follow the team's agreed coding style and linter rules.
    - If the disagreement still cannot be resolved using these criteria, the current code steward will make the final decision based on the available evidence, and the reasoning will be briefly documented in DECISIONS.md.
- Our definition of done for a piece of work
    - A task is considered complete when its required functionality has been implemented, tested, and reviewed by at least one other team member.

The code must be merged into the main branch, and important technical decisions must be documented in DECISIONS.md when applicable.

- How we communicate outside of class
    - We use LINE for communication and GitHub to track tasks and progress.
    - We discuss project updates at least once a week.
    - Team members are expected to respond to messages within 24 hours and inform the team in advance if they encounter problems or cannot meet a deadline.

2. Problem Statement with Evidence

A clear description of the collaboration pain point we are addressing.

- What the problem is, and who specifically experiences this problem
    - Our primary users are small research and software development teams (3-4 members) who use AI coding agents to work on GPU-intensive projects, such as machine learning, computer graphics, and scientific computing in a shared codebase and with limited computational resources. These teams often run interdependent experiments or tests, where changes or delays in one task may affect others. Without clear visibility into task dependencies, experiment progress, and GPU availability, team members may encounter scheduling conflicts, unnecessary waiting, or repeated work.
- Evidence from real users

  We have spoken with five people outside our team who actually experience this problem. Bellow is the summary:
    - **Interview A :** (Three students who have taken the AI Capstone Course)

In their final project, they had hands-on experience across the full lifecycle of Physical AI—from real-world data collection, policy learning, and ultimately to evaluation(in simulation). The mission was using a Franka robotic arm to perform cup stacking. The developing process must coordinate multiple stages, including data collection in Isaac Sim, policy training with LeRobot, and policy evaluation. These tasks have dependencies and compete for GPU resources, potentially causing delays or out-of-memory (OOM) errors when executed simultaneously. They often have to frequently check the working status by asking teammates through LINE or Discord. Another fatal problem is that the course instructor only gave them limited credits for the computational resource which makes the efficiency of collaboration much more important.

- **Interview B :** (Two students from ACM lab @ NYCU)
  They are conducting autonomous driving research, which involves policy training and evaluation on a shared lab server. Each student works independently with their own Claude Code agent, and they meet twice a week to discuss progress and experimental results. To share updates, they typically ask their agents to generate Markdown (.md) summary files and then manually inform each other where those files are located. However, this approach makes it difficult to maintain a unified view of ongoing tasks, experiment results, and dependencies between team members' work.
  Additionally, the lab's GPU resources are shared with other researchers, making resource availability unpredictable and requiring manual coordination to avoid conflicts.

- Why existing tools or approaches do not adequately solve it
  - Teams typically rely on:

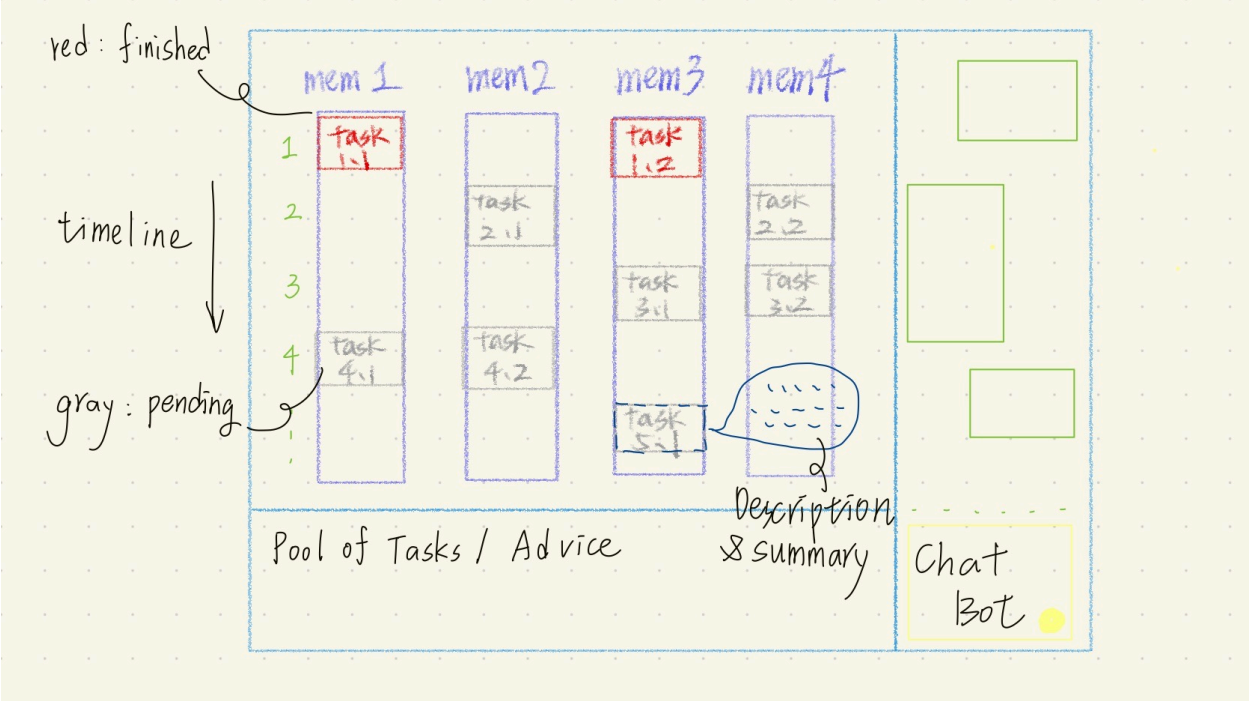
        - LINE, Discord, or informal discussions to coordinate tasks.
        - Git and shared directories to manage code changes.
        - Terminal commands (e.g., nvidia-smi) to check GPU availability.
        - Manual scheduling or informal agreements to share computing resources.

    However, these tools do not provide a unified view of task dependencies, experiment progress, and GPU availability. Team members must manually combine information from different sources, making it difficult to identify how changes or delays in one task affect other members' work.

3. Initial Interaction Design Sketch

- What the tool does at a high level
    - The tool will run on a shared machine and provide a web UI for managing tasks, tracking dependencies, and coordinating GPU usage. AI will help generate task suggestions, summarize progress, and identify potential conflicts or impacts on dependent tasks.
- What the inputs and outputs are
    - Inputs: Project requirements, upcoming work descriptions, task dependencies, GPU requirements, and progress updates.
    - Outputs: AI-generated task suggestions in a task pool, a shared timeline showing task assignments and progress, GPU usage information, dependency warnings, and AI-generated summaries.
- How two people would interact with it together
    - Member A enters the upcoming work into the task pool, and AI suggests tasks based on the description. Member A reviews the suggestions and drags selected tasks onto the timeline to assign them to team members. Member B can view the shared timeline and adjust their own tasks

accordingly. If a task is delayed or modified, the tool highlights potential impacts on dependent tasks and suggests adjustments. Both members can review the changes and coordinate their schedules.



4. Draft Evaluation Plan

- **Success definition:**
  - We will know our tool works if teams can coordinate dependent tasks and respond to GPU scheduling disruptions faster and with fewer errors than with their existing workflow.
  - Our metrics include differences in the following aspects:
    - Scheduling Errors: Number of conflicting GPU assignments or attempts to execute jobs before their dependencies are ready.

        - Task Awareness Accuracy: Whether members correctly identify teammates' responsibilities, task dependencies, and GPU job statuses.
        - Coordination Overhead: Number of clarification requests and manual status checks.
        - AI Suggestion Usefulness: Percentage of AI-generated task suggestions and dependency recommendations accepted by users without major corrections. For example, when a task is completed, the AI should correctly identify which dependent tasks can be performed next.
- **Target users and access:**
    - We will recruit at least one external research or development team, containing 3-4 members, and aim for two to three teams if possible. Potential participants include graduate students from our labs or students taking ML-related courses that require model experiments for their final projects.
- **Method and timing:**
    - We will conduct structured observations during Weeks 12–13.
    - Each team will complete two comparable experiment coordination scenarios: one using their existing workflow and one using our tool. The scenarios will include task dependencies and GPU scheduling disruptions.
    - For the structured observation, we will measure coordination time, scheduling errors, and task awareness accuracy.
    - Brief post-task interviews will be conducted to collect feedback on usability and AI suggestions.
- **Minimum evidence threshold:**

What would convince someone skeptical of your idea?

    - We will consider the evaluation promising if:
        - Both teams demonstrate faster coordination with the tool, with at least a 25% reduction in median coordination resolution time across teams.
        - Task awareness accuracy reaches at least 80% in the tool-assisted condition.
        - No critical GPU scheduling or experiment dependency conflict remains unresolved in the tool-assisted condition.
- **Human-AI boundary**
    - The AI will generate task suggestions, summarize experiment updates, identify potential cross-task impacts, and propose coordination actions based on available project records.
    - Deterministic logic will handle resource conflict checks, dependency validation, and experiment metadata tracking.
    - Human team members will remain responsible for assigning tasks, approving suggested dependency changes, choosing experimental configurations, deciding GPU allocation priorities, and authorizing job execution.
    - AI suggestions will be distinguishable from verified system states and will require human confirmation before modifying shared plans.
    - We will separately evaluate the AI component through the correctness and review cost of its task suggestions, summaries, and warnings.

5. Repository Link

https://github.com/MiniSnow0613/ai-in-the-loop-group22

Confirm that your GitHub repository is set up with:

- Branch protection on main

- PR template in place (`.github/PULL_REQUEST_TEMPLATE.md`)
- DECISIONS.md started with at least one entry
- Current code steward recorded in DECISIONS.md